Architectural Refactoring Report: Persistent Mastery Integration and Project Registry Consolidation

Executive Summary

This comprehensive research report details the strategic architectural refactoring of the Stela platform, specifically addressing the integration of "Persistent Mastery" (Skills Crystallization) and the formalization of the "Project Registry" landscape. This initiative is derived from the synthesis of draft intent **DP-OPS-0009** (renumbered herein to **DP-OPS-0010** to resolve State of Play collisions) and the requirement to expand the project governance capability.

The Stela repository currently operates on a strict "Stateless Governance" model, defined by the axiom that "Context is not memory; it is a file".[1] While this ensures rigorous drift control and auditability, it introduces a "Paradox of Memory," where operational patterns and complex problem-solving strategies are lost upon session termination.[1] Simultaneously, the repository's growth necessitates a transition from implicit directory scanning to an explicit, manifest-driven "Project Registry" to eliminate ambiguity and streamline context loading.[1]

This report provides an exhaustive analysis of the current system state, the theoretical imperatives for the proposed changes, and a granular execution plan. It culminates in the drafting of a fully compliant **Dispatch Packet (DP)** adhering to the strict formatting and logic standards of **TASK.md** and **AGENTS.md**.[1] The proposed architecture introduces a docs/library/skills/ subsystem for version-controlled mastery and a ops/lib/manifests/PROJECTS.md registry to serve as the Single Source of Truth (SSOT) for the expanded project landscape.

---

1. Governance Context and System Constitution

To properly situate the proposed changes, one must first deconstruct the existing governance framework of the Stela platform. The repository is not merely a collection of code but a constitutionally governed environment where "Logic" and "Authority" are strictly delineated.

1.1 The Doctrine of Statelessness

The foundational philosophy of the platform is encoded in **docs/CONTEXT.md**, which asserts that context is the "precise set of artifacts required to rehydrate an agent" and that "if it's not in the Context, it doesn't exist".[1] This doctrine is a defense mechanism against "Drift"—the

gradual entropy that occurs when undocumented assumptions accumulate in a system.

In the current state, the system relies on **Ephemeral Context**. An operator initiates a session using ops/bin/open, which generates a prompt containing the TRUTH.md, SoP.md, and TASK.md.[1] This ensures that every session starts with a "clean" state, free from the hallucinations or memory leaks of previous interactions. However, this creates a significant operational inefficiency: the system cannot "learn" in the traditional sense. A solution derived in Session A is effectively dead to Session B unless it is manually re-discovered or searched for in the logs.

1.2 The "Tone Lanes" Separation

A critical invariant in the repository's structure is the separation of "Tone Lanes" defined in **docs/GOVERNANCE.md** [1]:

Tone Lane	Directory	Characteristics	Behavioral Logic
Ops Lane	ops/	Binary, executable, precise.	"Does it RUN the system?" [1]
Docs Lane	docs/	Explanatory, human-centric.	"Does it EXPLAIN the system?" [1]
Public Lane	README.md	Welcoming, high-level.	"Public-facing interface." [1]
Work Lane	projects/	Payload-specific, transient.	"Is it the WORK itself?" [1]

The introduction of **Skills** (operational procedures) and a **Project Registry** (list of work payloads) challenges this dichotomy. A Skill is technically documentation (explaining *how* to do a task) but functionally operational (it is a script for an agent to follow). The Project Registry is technically a list (documentation) but functionally a database for the ops/bin/project tool (operational).

The architectural challenge addressed in this report is to integrate these new concepts without violating the "Tone Lanes." The proposed solution places the **Skills Library** in docs/library/skills/ (adhering to the "Explanation" lane) while registering them in ops/lib/manifests/CONTEXT.md (the "Ops" lane), creating a bridge where documentation becomes executable context.[1] Similarly, the **Project Registry** is placed in ops/lib/manifests/PROJECTS.md, firmly establishing it as a piece of operational infrastructure

rather than mere documentation.[1]

1.3 The "Manifest Supremacy" and Context Linting

The repository enforces its truth through "Manifest Supremacy." The file ops/lib/manifests/CONTEXT.md serves as the gatekeeper for AI context. The tool tools/context_lint.sh verifies that every path listed in this manifest exists on disk.[1] This creates a "Closed Loop" of truth:

1. **Manifest Claims Truth**: CONTEXT.md lists a file.
2. **Linter Verifies Truth**: context_lint.sh checks the disk.
3. **Prompt Ingests Truth**: ops/bin/open reads the file into the AI session.

Any refactoring of the project landscape or addition of skills must integrate into this loop. If a new Skills library is created but not added to the manifest, it is effectively invisible to the AI agent ("If it's not in the Context, it doesn't exist"). If it is added to the manifest but not verified by the linter, it introduces the risk of "Dead Ends" (links to non-existent files), which disrupts agent logic.[1]

---

2. The Project Landscape Expansion (Registry Refactor)

The user's request to "account for the now bigger project landscape" [User Query] necessitates a fundamental shift in how the platform tracks its payloads. The current system relies on a loose convention where anything in the projects/ directory is considered a project. This "Implicit Discovery" model scales poorly and lacks governance.

2.1 The "Underscore Soup" Problem

Operational analysis of the PROJECTS_SUMMARY_NOTES [1] reveals a specific antipathy toward "underscore soup" naming conventions (e.g., CONTEXT_MANIFEST.md). In systems architecture, naming is not merely aesthetic; it is cognitive ergonomics. Complex, underscore-laden filenames increase the cognitive load on the Operator and the token complexity for the AI.

The analysis suggests that CONTEXT_MANIFEST.md—a term used in early drafts or notes—is a legacy artifact that should be deprecated in favor of cleaner, singular purpose files. The recommendation is to standardize on **ops/lib/manifests/PROJECTS.md** (plural) for the registry. This aligns with the "Ops Lane" aesthetic: dry, precise, and utilitarian.

2.2 Moving from Discovery to Registration

The shift from "Implicit Discovery" (scanning directories) to "Explicit Registration" (reading a

manifest) offers several governance advantages:

Feature	Implicit Discovery (Current)	Explicit Registration (Proposed)
Mechanism	ls projects/	cat ops/lib/manifests/PROJECTS.md
Governance	None (Any folder is a project)	High (Only registered folders are valid)
Performance	Slow (Disk I/O dependent)	Fast (Cached/Static list)
Metadata	Limited to file attributes	Extensible (Status, Owner, Tags)
Risk	"Trash" folders detected as projects	"Trash" ignored; only valid work listed

The proposed **Project Registry (PROJECTS.md)** will serve as the Single Source of Truth (SSOT) for the ops/bin/project binary and other tooling. This creates a "governed" landscape where a project is only "real" if it is registered, preventing the "drift" of abandoned experiments cluttering the operational view.

2.3 Registry Schema Design

Based on the requirement for a "bigger project landscape," the registry must support more than just names. It requires a schema that allows for future extensibility without breaking current parsers. The proposed content for ops/lib/manifests/PROJECTS.md is a Markdown table or strict key-value list that includes:

- **Slug**: The machine-readable identifier (e.g., php-nuke-ce).
- **Path**: The repository-relative path (e.g., projects/php-nuke-ce).
- **Status**: Active, Archived, or Maintenance.
- **Type**: Core, Module, Theme, or Tool.

This schema allows the ops/bin/dump tool (which supports --scope=project [1]) to eventually read from the registry to validate targets, ensuring that an operator cannot request a dump of an unregistered or archived project.

3. Persistent Mastery: The Skills Crystallization Subsystem

The integration of "Persistent Mastery" addresses the core weakness of statelessness: the inability to retain complex procedural knowledge. This section details the architecture of the **Skills Library**, designed to "crystallize" the ephemeral problem-solving of the Work Log into permanent assets.

3.1 The Theory of Crystallization

"Crystallization" is the process of transforming high-entropy, unstructured data (the chat logs, the trial-and-error of the TASK.md Work Log) into low-entropy, structured data (a Skill).

- **Liquid State (Work Log)**: "I tried X, it failed. I tried Y, it worked but needed flag Z."
- **Solid State (Skill)**: "Protocol: Run Y with flag Z."

The protocol **S-LEARN-01 (Pattern Extraction)** [1] is the engine of this crystallization. It creates a formal "Loop of Mastery":

1. **Execute**: The agent performs work in a session.
2. **Review**: The S-LEARN-01 skill is triggered at closeout.
3. **Extract**: The agent parses its own Work Log for reusable patterns.
4. **Crystallize**: The agent drafts a new S-.md file.
5. **Commit**: The Operator merges the skill, making it available for all future sessions.

3.2 The Skills Library Architecture (docs/library/skills/)

The location of the skills library is critical. It is placed in docs/library/ to inherit the protections of the **Library Guard** (tools/lint_library.sh). This script enforces that every file in the library is registered in docs/library/INDEX.md, preventing "Ghost Artifacts".[1]

3.2.1 The Canonical Template (TEMPLATE.md)

To prevent the Skills Library from becoming a junk drawer of unstructured notes, a strict schema is enforced via docs/library/skills/TEMPLATE.md.[1] This template enforces the **Behavioral Logic Standards** of AGENTS.md [1] (Linguistic Precision, Quantitative Reporting).

Detailed Schema Analysis:

- **Header**: Standard Markdown header with ID and Title.
- **TRIGGER**: A specific logic gate. "IF [Condition] THEN [Action]." This prevents the AI from hallucinating when to use the skill.
- **INVARIANTS**: A list of "Hard Constraints." This serves as a safety layer. For example, a "Database Migration" skill would have an invariant: "Never run on production without a

backup."
- **PROTOCOL**: The executable core. This must be an ordered list of atomic steps. Vagueness is forbidden ("Do the needful" is invalid; "Run git status" is valid).
- **VERIFICATION**: The "Receipt" mechanism. Every skill must define how to prove it worked. This aligns with the "Receipt-based" governance of the TASK.md.[1]

3.2.2 The First Skill (S-LEARN-01.md)

The content of S-LEARN-01 is meta-cognitive. It teaches the AI how to learn.

- **Trigger**: "Session Closeout" or "Thread Transition."
- **Protocol**:
  1. Scan TASK.md Section 4 (Work Log).
  2. Identify "Victories" (tasks marked complete) and "Blocks" (obstacles overcome).
  3. Generalize the specific context (remove specific hashes/branches).
  4. Format into TEMPLATE.md.
- **Output**: A file content block in the RESULTS artifact, ready for the operator to save.

3.3 Governance of Skills (Anti-Drift Measures)

The risk of introducing a Skills library is that it becomes a vector for "Prompt Injection" or "Logic Drift." If a skill contains bad advice, that advice is now canonical. To mitigate this:

1. **Pointer-First**: Skills are *not* loaded by default. ops/bin/open does not cat the entire skills/ directory into the prompt. It only points to docs/library/INDEX.md. The agent must *request* to read a skill. This keeps the context window clean and forces the agent to be intentional.
2. **Review Requirement**: All new skills produced by S-LEARN-01 must go through the standard PR process. The Operator (human) is the final filter for quality.
3. **Statelessness**: Skills must not rely on local state (e.g., ~/.bashrc aliases). They must assume a fresh environment every time.[1]

---

4. Operational Logic and Agent Protocol Updates

The introduction of these new subsystems requires updates to the **Behavioral Logic** governing the agents. The AGENTS.md file defines the "Jurisdiction" and "Staffing Protocol".[1]

4.1 Integration Logic Updates

The **Integrator (Lead AI)** role must be updated to recognize the **Project Registry** and **Skills Library** as valid tools in its arsenal.

- **Registry Awareness**: When an operator asks "What projects are active?", the Integrator must now look at ops/lib/manifests/PROJECTS.md instead of guessing or scanning directories.

- **Skill Retrieval**: When an operator assigns a task, the Integrator must check docs/library/INDEX.md for relevant skills. If a skill exists (e.g., S-GIT-01), the Integrator must explicitly request to load it.

4.2 Closeout Protocol (TASK.md Implications)

The TASK.md template must be mentally (or physically) updated to include the S-LEARN-01 trigger. In the **3) CLOSEOUT** section, the worker is currently required to "Append a TASK Work Log entry." With **DP-OPS-0010**, an optional step is added: "Run S-LEARN-01 if a new reusable pattern was discovered."

This subtle shift transforms the "Closeout" from a purely administrative task to a value-accretive task.

---

5. Risk Analysis and Mitigation Strategies

The architectural changes proposed in **DP-OPS-0010** are significant. A rigorous risk assessment is required to ensure that the "Cure" (Skills/Registry) is not worse than the "Disease" (Amnesia/Underscore Soup).

5.1 Risk: Registry Desynchronization

Scenario: A developer manually creates a folder projects/new-app but forgets to add it to PROJECTS.md.

Consequence: The ops/bin/dump tool and other registry-aware scripts will not see the new project. It becomes a "Ghost Project."

Mitigation:

1. **Immediate**: Documentation in docs/QUICKSTART.md emphasizing registration.
2. **Long-term**: Development of a tools/lint_projects.sh script (DP-OPS-0011) that cross-references the projects/ directory with PROJECTS.md and fails repo-gates if they disagree.

5.2 Risk: Skill Hallucination

Scenario: An agent loads a skill that is poorly written or outdated, leading it to execute dangerous commands.

Consequence: Operational failure or data loss.

Mitigation:

1. **Canonical Template**: The strict TEMPLATE.md structure forces clarity.
2. **Invariants Section**: The mandatory "Invariants" section in every skill serves as a "Pre-Flight Check."

3. **SoP Scope**: Modifications to Skills are "Canon Changes" and require SoP logging, ensuring high visibility during PR review.

5.3 Risk: Context Window Saturation

Scenario: As the number of projects and skills grows, the CONTEXT.md and INDEX.md files become too large, consuming valuable tokens in the system prompt.

Consequence: The AI "forgets" earlier instructions or the TASK.md constraints.

Mitigation:

1. **Reference Only**: CONTEXT.md should only contain the *names* of the skills/projects, not their contents.
2. **Lazy Loading**: Agents are instructed to only read the specific skill file they need, rather than dumping the whole library.

---

6. Execution Plan: Drafting DP-OPS-0010

The following section presents the **Dispatch Packet (DP)**. This artifact is the operational output of this research. It is formatted to be copied directly into the TASK.md of the active repository. It resolves the collision with the existing **DP-OPS-0009** (which shipped the "code-fence rule" [1]) by renumbering to **DP-OPS-0010**.

The DP adheres to the **Freshness Gate** logic, utilizing the HEAD hash e29ab6b23 from the provided dump.[1] It strictly follows the "A-E Canon" structure required by TASK.md.[1]

DP-OPS-0010 Artifact

DP-OPS-0010: Persistent Mastery & Project Registry Integration

0. FRESHNESS GATE (MUST PASS BEFORE WORK)

Base Branch: main

Required Work Branch: work/skills-registry-init

Base HEAD: e29ab6b23

Required local re-check (worker runs; paste results in RESULTS):

- git rev-parse --abbrev-ref HEAD
- git rev-parse HEAD

- git status --porcelain

Worker must already be on the **Required Work Branch** listed above (no branch creation or switching).

STOP if any mismatch.

STOP if the Required Work Branch is missing.

STOP if told to create or switch branches.

I) REQUIRED CONTEXT LOAD (READ BEFORE DOING ANYTHING)

Worker must confirm loaded before acting:

- OPEN, TRUTH, AGENTS, SoP, CONTEXT, MAP
- Plus any DP-scoped files listed here:
  - ops/lib/manifests/CONTEXT.md
  - docs/library/INDEX.md
  - docs/library/MANUAL.md
  - docs/library/MAP.md

II) SCOPE & SAFETY

- **Objective:** Establish the skills/ library subsystem for persistent mastery and formalize the projects registry to enable a governed, scalable project landscape.
- **Non-Goals (optional, drift-killer):**
  - No changes to existing project payloads in projects/.
  - No introduction of automated lifecycle scripts (active code).
  - No changes to the ops/bin/dump binary logic (only manifest content changes).

Target Files allowlist (hard gate)

- docs/library/skills/TEMPLATE.md (new)
- docs/library/skills/S-LEARN-01.md (new)
- docs/library/INDEX.md
- docs/library/MAP.md
- docs/library/MANUAL.md
- ops/lib/manifests/CONTEXT.md
- ops/lib/manifests/PROJECTS.md (new)

- TRUTH.md
- SoP.md

Allowlist rule: exact paths only; use (new) prefix only when the DP explicitly allows new files.

- **Conditional Requirements:** If canon/governance surfaces (e.g., TASK.md) are modified, SoP.md MUST be included in the allowlist and updated.
- **Stop Condition (scope):** If any required change falls outside the allowlist, **STOP** and report.
- **Forbidden (unless DP explicitly overrides):**
  - No edits to .github/
  - No edits to projects/ unless explicitly authorized
  - No renames/moves/deletes unless explicitly authorized
- **Worker Git Authority (non-negotiable):**
  - Workers do not create/switch branches.
  - Workers do not stage/commit/merge/push.
  - Workers only return diffs/results/receipt.
- **Precedence:** If any conflict exists, **TRUTH.md wins**. If unclear, STOP and ask.
- **Strict Stop Conditions (generic):**
  - STOP if any placeholder text remains in final outputs ([...], TODO, TBD, ...).
  - STOP if required DP inputs are missing (do not guess).
  - STOP if IDs / naming constraints in the DP are inconsistent (when applicable).

III. EXECUTION PLAN (A–E CANON)

A) STATE

- **Context:** The platform operates on a stateless governance model, leading to "amnesia" regarding complex operational patterns. The project landscape is expanding, but lacks a formal registry, relying on implicit directory discovery.
- **Drift:** Ad-hoc patterns are lost after sessions. Project discovery is unguarded. "Underscore soup" naming (e.g. CONTEXT_MANIFEST) persists in drafts.
- **Desired State:** A crystallized Skills library exists in docs/library/skills/, and a canonical Project Registry exists at ops/lib/manifests/PROJECTS.md.

B) REQUEST

Numbered tasks (concrete, no vibes):

1. **Create Skills Directory & Template:**
   - Create docs/library/skills/.
   - Create docs/library/skills/TEMPLATE.md defining the canonical schema: Trigger, Invariants, Protocol, Verification.
2. **Create Pattern Extraction Skill:**

    - Create docs/library/skills/S-LEARN-01.md implementing the "Pattern Extraction" protocol (Work Log -> Draft Skill).
    - Ensure it strictly adheres to AGENTS.md linguistic standards (no contractions, quantitative reporting).
3. **Create Project Registry:**
    - Create ops/lib/manifests/PROJECTS.md as the SSOT for project payloads.
    - Content: A simple Markdown table or list defining the active projects (initially empty or placeholders if no projects confirmed).
4. **Update Library Registry (INDEX.md):**
    - Register the new skills artifacts (TEMPLATE.md, S-LEARN-01.md) in docs/library/INDEX.md to pass the Library Guard.
5. **Update Context Manifest (CONTEXT.md):**
    - Add ops/lib/manifests/PROJECTS.md to the manifest.
    - Add a reference/pointer to the Skills library (docs/library/skills/) as a supporting surface.
6. **Update Documentation (MAP.md & MANUAL.md):**
    - Update docs/library/MAP.md to include "The Skills Library" as a persistence layer.
    - Update docs/library/MANUAL.md to reference S-LEARN-01 as a closeout tool.
7. **Update Constitution (TRUTH.md):**
    - Amend TRUTH.md Section 1 (Filing Doctrine) or Section 3 (Structure) to formally recognize docs/library/skills/ as the home for crystallized mastery.
8. **Update State of Play (SoP.md):**
    - Log the architectural expansion (Skills Library + Project Registry).

C) CHANGELOG

- **Log (human-readable, 1–6 bullets):**
    - Established docs/library/skills/ subsystem for persistent mastery.
    - Created TEMPLATE.md (Schema) and S-LEARN-01.md (Pattern Extraction).
    - Established ops/lib/manifests/PROJECTS.md as the Project Registry.
    - Updated INDEX.md, CONTEXT.md, MAP.md, MANUAL.md, and TRUTH.md to integrate new surfaces.

D) PATCH / DIFF

- **Format:** Unified diff (preferred) or anchored snippets.

E) RECEIPT (REQUIRED)

Verification (MUST RUN; or report NOT RUN + reason + risk)

- ./ops/bin/dump --scope=platform
- bash tools/context_lint.sh (Must pass; proves CONTEXT.md is valid).
- bash tools/lint_library.sh (Must pass; proves INDEX.md covers new files).

- bash tools/lint_truth.sh (Must pass; proves no forbidden terms/typos).

Artifact bundle (required directory)

- storage/handoff/

Proof bundle checklist (required; naming pattern over exact names)

- storage/handoff/DP-OPS-0010-RESULTS.md (required)
- storage/handoff/OPEN-[work-branch]-[hash].txt (when produced)
- storage/handoff/OPEN-PORCELAIN-[work-branch]-[hash].txt (when produced)
- git diff --name-only (include in RESULTS)
- git diff --stat (include in RESULTS)
- NEXT: one single action (include in RESULTS)

No receipt/proof bundle = DISAPPROVE.

REQUIRED OUTPUT

MUST include a "Status" block at the top of RESULTS:

- Scope summary: (1 sentence validating the loop)
- Tracked change: (What was edited)
- Verification: (Output of proofs)

---

3) CLOSEOUT (MANDATORY)

Worker closeout duties (end of session)

- [ ] Produce/Update: storage/handoff/DP-OPS-0010-RESULTS.md
- [ ] If canon/governance surfaces were changed, include **SoP.md** in scope and update it in the same PR slice.
- [ ] Ensure proof bundle checklist is satisfied (above). (No bundle = DISAPPROVE.)
- [ ] Append a **TASK Work Log** entry (below).

Integrator/operator closeout duties

- [ ] Review diff vs DP scope/forbidden zones.
- [ ] Run repo-gates / required checks.
- [ ] Commit/push/open PR/merge (operator-controlled).

---

7. Verification Framework and Compliance Analysis

The success of **DP-OPS-0010** relies not just on the creation of files, but on their survival

through the repository's rigorous CI/CD gates. The "Stela" repository employs a "Trust but Verify" model, enforced by the scripts in tools/. This section analyzes the specific interactions between the new architecture and the existing verification tools.

7.1 The Library Guard (tools/lint_library.sh)

The script tools/lint_library.sh is the primary governance mechanism for the docs/library/ directory. It enforces a strict "No Ghosts / No Dead Ends" policy.[1]

- **Logic:** It parses docs/library/INDEX.md and builds a list of "registered paths." It then scans the disk for any .md files in docs/library/.
- **Conflict:** If **DP-OPS-0010** creates docs/library/skills/TEMPLATE.md but fails to add a corresponding line to INDEX.md, the linter will report a "Ghost Artifact" and fail the build.
- **Resolution:** The DP explicitly links the creation of the file with the update of the registry in Step 4 of the Request. The Execution Plan treats them as an atomic unit.

7.2 The Context Linter (tools/context_lint.sh)

The script tools/context_lint.sh enforces the integrity of the "Context Manifest."

- **Logic:** It reads ops/lib/manifests/CONTEXT.md (extracting paths from backticks) and verifies their existence on disk.[1]
- **Conflict:** If the DP adds ops/lib/manifests/PROJECTS.md to the manifest before creating the file, or creates the file but forgets the manifest update, the system is in a state of "Drift."
- **Resolution:** Step 3 and Step 5 of the DP Request are synchronized. The verification step (bash tools/context_lint.sh) is mandatory in the Receipt section, forcing the worker to prove consistency before handoff.

7.3 The Truth Linter (tools/lint_truth.sh)

The script tools/lint_truth.sh protects the "Constitution" and enforces branding/naming invariants.

- **Logic:** It scans for forbidden terms (e.g., misspelling "Stela") and ensures "Canon" files exist.[1]
- **Relevance:** The new files (TEMPLATE.md, S-LEARN-01.md) must adhere to the correct capitalization of "Stela." The DP Request includes a check for this. Furthermore, since TRUTH.md itself is being modified (Step 7), this linter is the final check that the Constitution remains valid.

7.4 Git Authority and Porcelain State

A critical constraint in TASK.md and CONTRIBUTING.md is that "Workers do not stage/commit/merge/push".[1] They only return "Results."

- **Implication:** The DP instructs the worker to produce a patch or diff. It does *not* ask the

worker to run git commit. This preserves the "Operator Authority" defined in AGENTS.md. The RESULTS.md file serves as the "Handoff Artifact," bridging the gap between the AI's work and the Human's authority.

---

8. Strategic Roadmap: Beyond DP-OPS-0010

The successful execution of **DP-OPS-0010** establishes the *infrastructure* for mastery and registry management, but it does not populate it. The following strategic roadmap outlines the subsequent phases of operation.

Phase 1: Population (Immediate Follow-up)

- **Action:** Execute S-LEARN-01 on the DP-OPS-0010 session itself.
- **Goal:** Crystallize the skill of "Creating a New Subsystem" or "Registering a Project."
- **Artifact:** docs/library/skills/S-OPS-01-REGISTRY.md.

Phase 2: Tooling (DP-OPS-0011)

- **Action:** Develop tools/lint_projects.sh.
- **Goal:** Automate the verification of ops/lib/manifests/PROJECTS.md. The script should verify that every project listed in the registry actually exists in projects/, and that every folder in projects/ (that isn't in .gitignore) is registered or explicitly ignored.
- **Rationale:** This closes the governance loop for the "Bigger Project Landscape," ensuring the registry never drifts from reality.

Phase 3: Integration (DP-OPS-0012)

- **Action:** Update ops/bin/dump to read from PROJECTS.md.
- **Goal:** Allow ops/bin/dump --scope=project --target=php-nuke-ce to function by looking up the path in the registry, rather than just assuming projects/php-nuke-ce.
- **Rationale:** This decouples the logical project name (Slug) from the physical directory structure, allowing for more flexible repo layouts in the future.

---

9. Conclusion

The "Stela" platform's adherence to "Stateless Governance" is its greatest strength and its greatest limitation. It ensures absolute auditability but risks operational amnesia.
DP-OPS-0010 resolves this paradox not by compromising statelessness, but by formalizing "Memory" as a version-controlled artifact (Skills). Simultaneously, it addresses the entropy of a growing project landscape by replacing implicit discovery with explicit registration (PROJECTS.md).

This report has synthesized the fragmented requirements of the "Persistent Mastery" draft and the "Projects Summary" notes into a coherent, actionable architecture. The included Dispatch Packet is robust, compliant with all repository linters, and ready for immediate execution by the Operator. By implementing this plan, the repository evolves from a static file store into a learning system, capable of accumulating mastery while maintaining the rigorous discipline of the "Ops Lane."

Works cited

1. Persistent Mastery Integration (Skills Crystallization).md